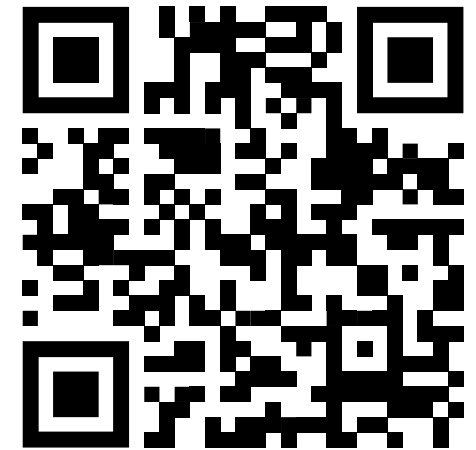


Programmierung in C++

Vererbung

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Einführung

Lernziele

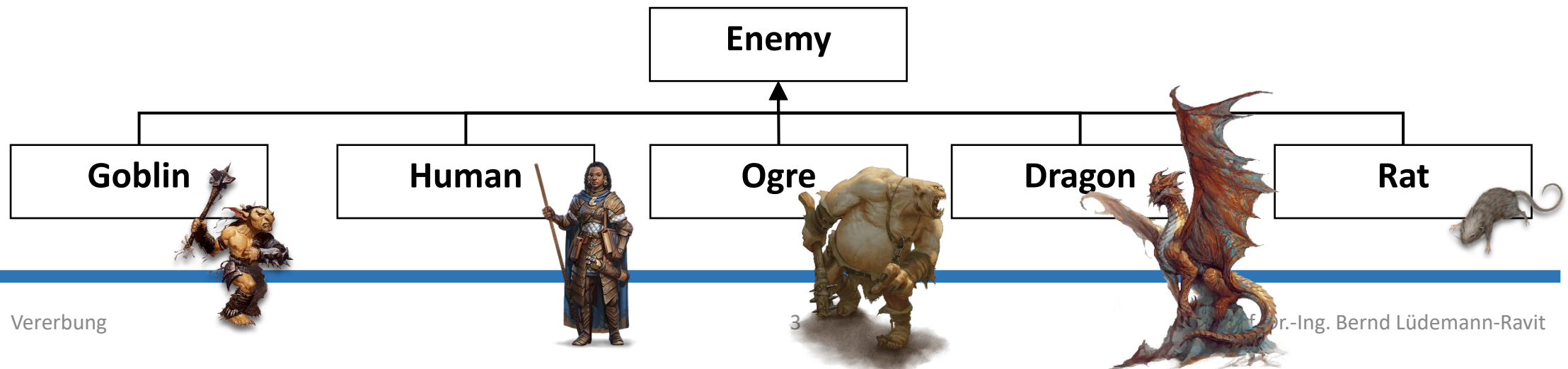
- Der Nutzen und das Prinzip von Vererbung wurden verstanden
 - Dazu wird ein durchgehendes Beispiel mit verschiedenen Gegner-Typen verwendet.
- Die unterschiedlichen Zugriffs-Spezifikation im Kontext von Vererbung sind bekannt

Einführung

Generalisierung & Spezialisierung



- **Gemeinsame** Attribute oder Methoden mehrerer Klassen können in eine übergreifende Klasse, die **Elternklasse**, ausgelagert werden (**Generalisierung**)
 - Gemeinsame Methoden müssen nicht mehrfach implementiert und bei Änderungen angepasst werden, sondern können einmalig in der Elternklasse definiert werden.
- **Unterschiedlichen** Attribute, Methoden und Methoden-Implementationen bleiben in den **Kindklassen** (**Spezialisierung**)
 - Instanzen der Kindklasse werden immer auch als Instanzen der Elternklasse aufgefasst
 - Jede Spezialisierung kümmert sich um genau das, was sie auszeichnet und somit von den anderen Spezialisierungen unterscheidet.



Einführung

Zugriffs-Spezifikationen

- Member, die als **public** gekennzeichnet sind, sind überall sichtbar, in denen die Header-Datei inkludiert wird
- Kindklassen **erben** (Inheritance) alle Attribute und Methoden von ihrer Elternklasse und können auf diejenigen zugreifen, die **nicht private** sind
 - Member, die als **private** deklariert sind, sind im Scope der Kindklassen „unsichtbar“
- Sollen Attribute und Methoden nur für die Kindklassen, aber nicht außerhalb der Klassen sichtbar sein, so werden sie mit dem Schlüsselwort **protected** deklariert.



```
class Enemy {  
    // Konstruktor, Destruktor, Getter, Setter  
    public:  
        explicit Enemy(std::string name, int health);  
        /* ... */  
        std::string getName() const;  
        int getHealth() const;  
    // sichtbare Methoden  
    public:  
        void injure(int damage);  
    // Hilfs-Methoden  
    private: // nur im Scope von Enemy bekannt  
        void calculateActualDamage(int damage);  
    // Attribute  
    protected: // auch sichtbar für erbende Klassen  
        std::string m_name;  
        int m_health;  
};
```

Einführung

Zugriffs-Spezifikationen



- Erbende Klassen werden in der Deklaration nach der Benennung durch einen **Doppelpunkt**, einer Zugriffs-Spezifikation und dem Namen der Elternklasse gekennzeichnet.

Ist ein Member in „Enemy“	public	protected	private
... wird er in „Rat“ (private)	private	private	unsichtbar
... wird er in „Ogre“ (protected)	protected	protected	unsichtbar
... wird er in „Dragon“ (public)	public	protected	unsichtbar

```
class Enemy {  
public:  
    explicit Enemy(std::string name, int health);  
    /* ... */  
    void injure(int damage);  
private:  
    void calculateActualDamage(int damage);  
protected:  
    std::string m_name;  
    int m_health;  
};
```

```
class Rat : private Enemy {  
    // ...  
};
```

```
class Ogre : protected Enemy {  
    // ...  
};
```

```
class Dragon : public Enemy {  
    // ...  
};
```

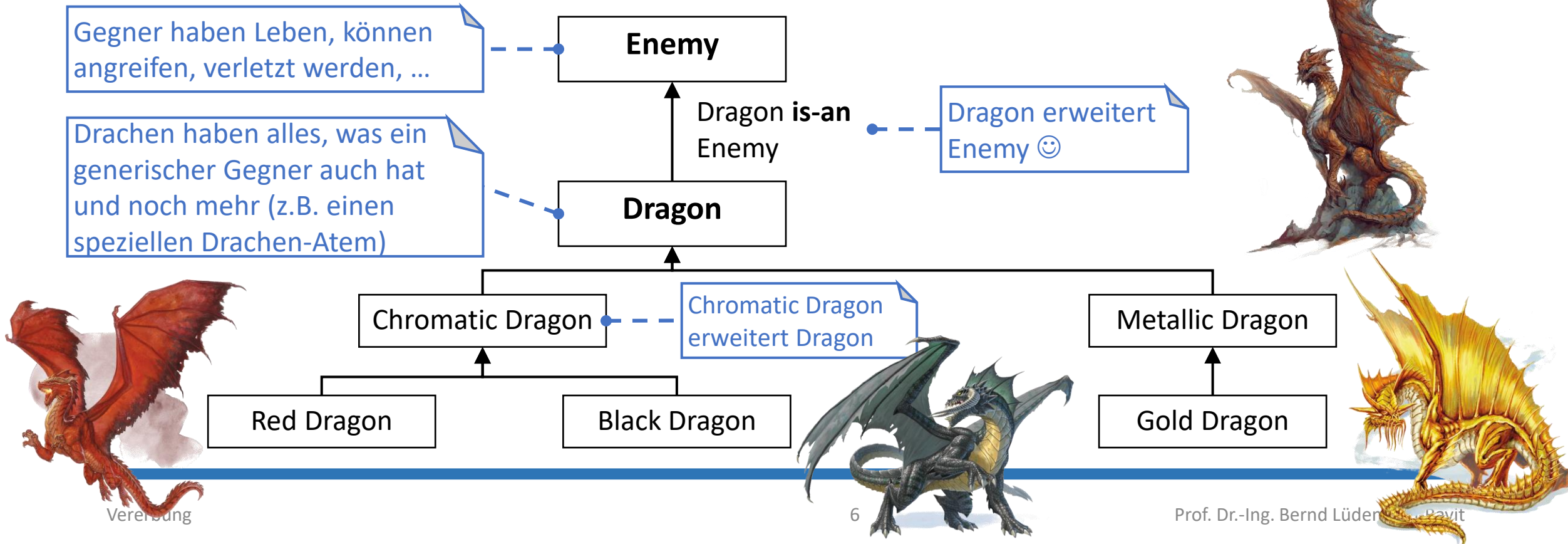
Einführung

Eltern-Kind-Beziehung



■ Public Inheritance ist eine „is-a“-Beziehung

- Alles, was auf eine Elternklasse zutrifft, muss auch auf die Kindklasse zutreffen
- **Kindklassen erweitern und spezifizieren** die Eigenschaften von Elternklassen und schränken sie nicht ein



Einführung

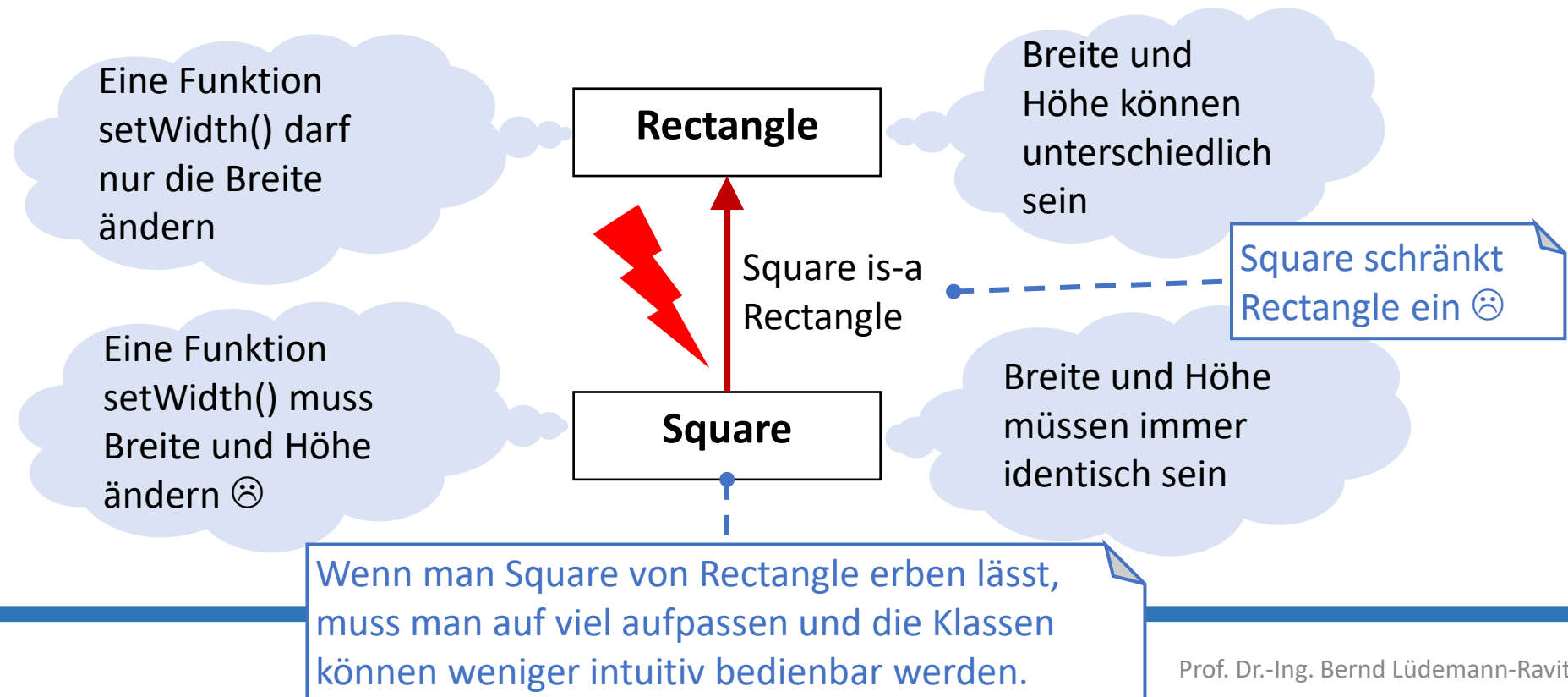
Eltern-Kind-Beziehung



- Public Inheritance ist eine „is-a“-Beziehung
 - Die in der Schule entwickelte Intuition kann einer geeigneten Implementierung widersprechen

Negativbeispiel:

Mathematische Formen sind ein beliebtes Anfängerbeispiel für Vererbung, um grundlegende Funktionalitäten zu zeigen. Aber in der Implementierung ist ein Square eigentlich gar kein Rectangle.



Einführung

Objekt-Erzeugung



- Der Konstruktor einer Kindklasse muss immer auch das Eltern-Objekt konstruieren
 - Besitzt die Elternklasse einen Standard-Konstruktor, wird dieser implizit aufgerufen
 - Ansonsten muss ein passender Konstruktor der Elternklasse explizit aufgerufen werden

```
class Enemy {  
public:  
    explicit Enemy(std::string name, int health);  
    /* ... */  
};
```

↑ erbt von

```
#include "Enemy.h"  
class Ogre : public Enemy {  
public:  
    explicit Ogre(std::string name, int health,  
                  int durabilityClub);  
    /* ... */  
}
```

```
#include "Enemy.h"  
Enemy::Enemy(std::string name, int health)  
    : m_name(name), m_health(health)  
{ }
```

↑ ruft auf

```
#include "Ogre"  
Ogre::Ogre(std::string name, int health, int durabilityClub)  
    : Enemy(name, health)  
    , m_durabilityClub(durabilityClub)  
{ }
```

Ogre muss **als erstes** den Konstruktor der Elternklasse aufrufen!

Einführung

Zusammenfassung



- Durch **Vererbung** können Attribute und Methoden, die mehrere Klassen gemeinsam haben, in einer **Elternklasse** zusammengefasst werden.
 - Die Kindklassen **spezialisieren** das unterschiedliche Verhalten zwischen den Klassen.
 - Kindklassen stehen zu ihrer Elternklasse in einer „**is-a**“-**Beziehung**
 - Kindklassen besitzen immer die „Fähigkeiten“ und das „Wissen“ der Elternklasse, die in ihr **nicht als private** deklariert sind.
- Durch eine **Zugriffsspezifikation** bei der Vererbung kann angegeben werden, mit welchen Zugriffsrechten die Member der Elternklasse vererbt werden
 - Ohne Angabe wird implizit **private** spezifiziert
 - In der Regel verwendet man immer die **public** Zugriffsspezifikation
- Eine Instanz einer erbenden Klasse ruft immer den **Konstruktor** der Elternklasse auf
 - Ist kein Standard-Konstruktor für die Elternklasse vorhanden, muss der Konstruktor-Aufruf explizit in dem Initialisierungs-Bereich des Kindklassen-Konstruktors stattfinden

Einführung

Verständnisfragen

PIN: 967140

<https://poll.hs-kempten.de/poll/>



- **Wie müssen Member-Variablen deklariert sein, damit sie erbenden Klassen zur Verfügung stehen?**
 - private
 - protected
 - public
 - static
- **WAHR oder FALSCH?**
 - Der Wert von Member-Variablen, die als **protected** deklariert sind, kann von außerhalb der Klasse erhalten, aber nicht geändert werden.
 - Instanzen einer Kindklasse können alle Methoden der Elternklasse aufrufen, die nicht mit **private** deklariert sind.



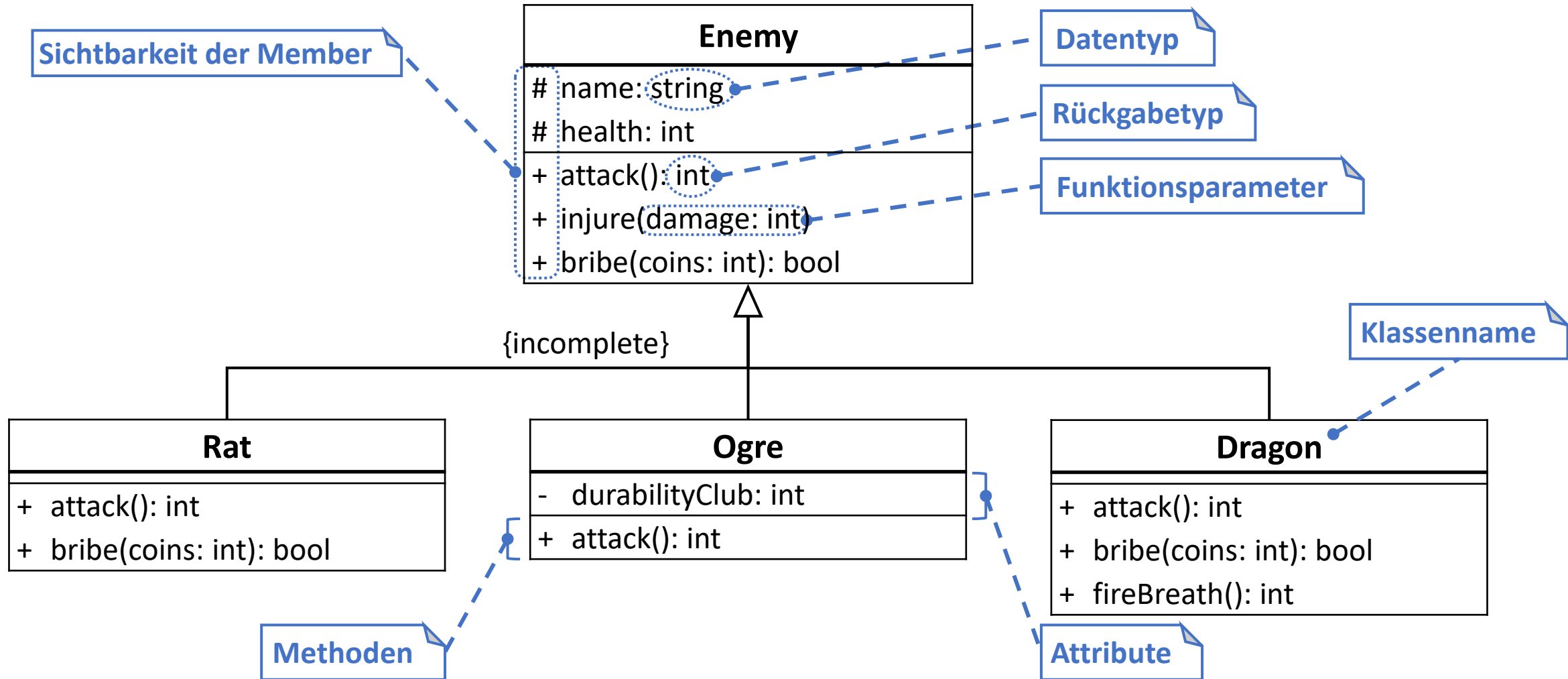
UML-Klassendiagramm

Lernziele

- UML-Klassendiagramme können als Abstraktion verwendet werden, um Klassen und Vererbung visuell zu verdeutlichen.
- Aus vorgegebenen UML-Klassendiagrammen können C++ Klassen implementiert werden.

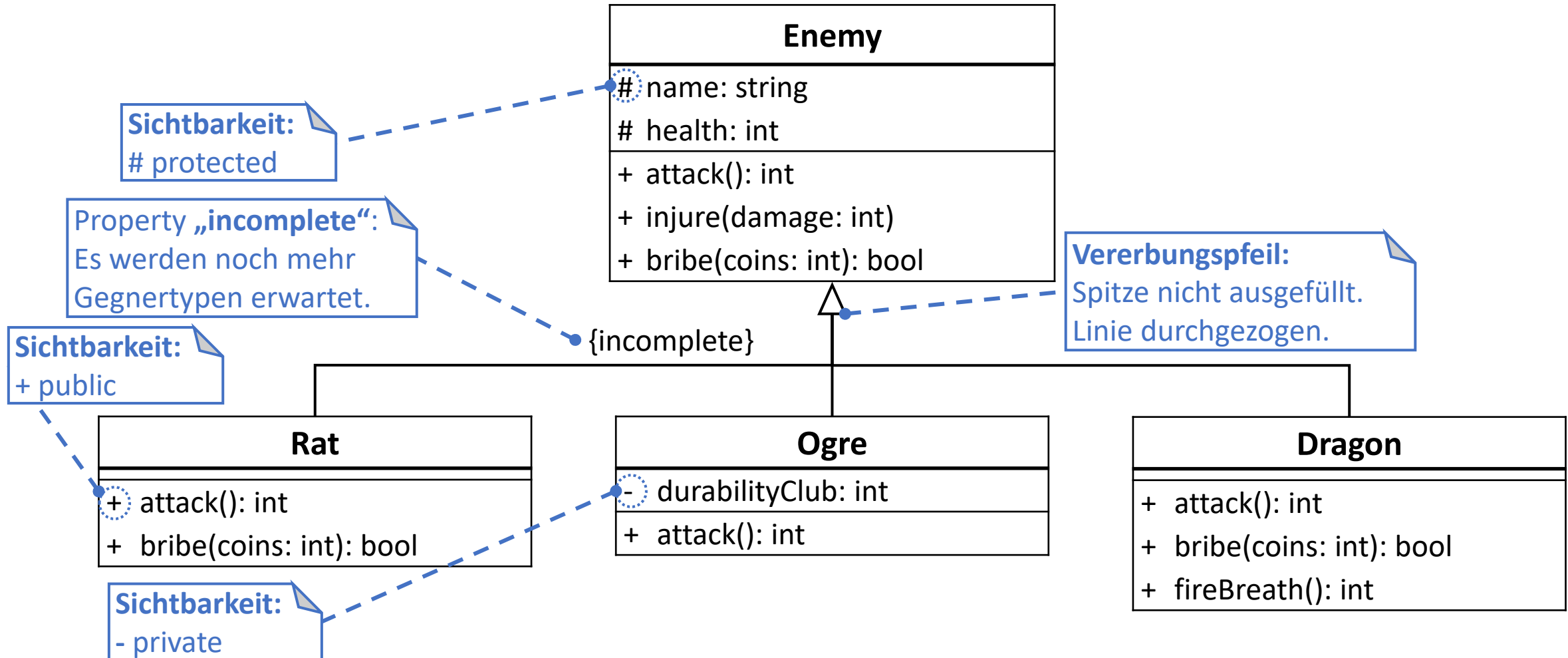
UML-Klassendiagramm

Methoden & Attribute



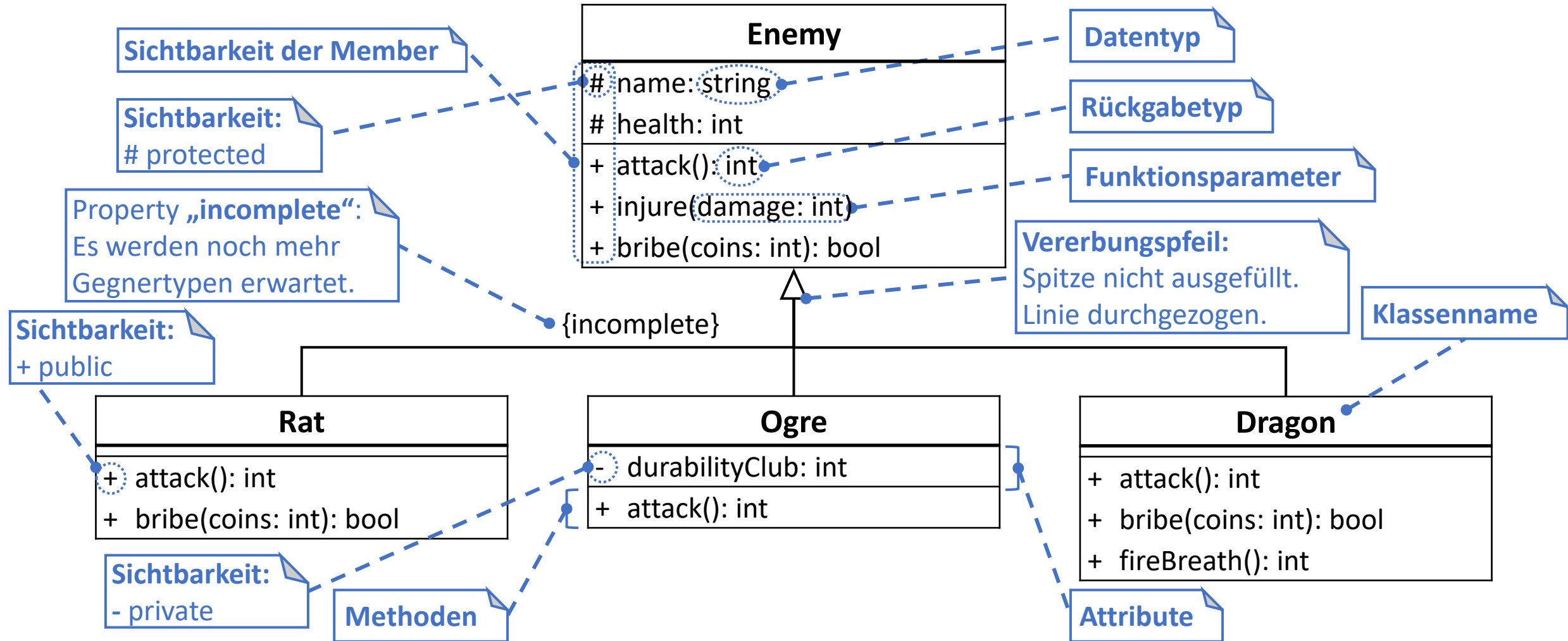
UML-Klassendiagramm

Sichtbarkeit & Vererbung



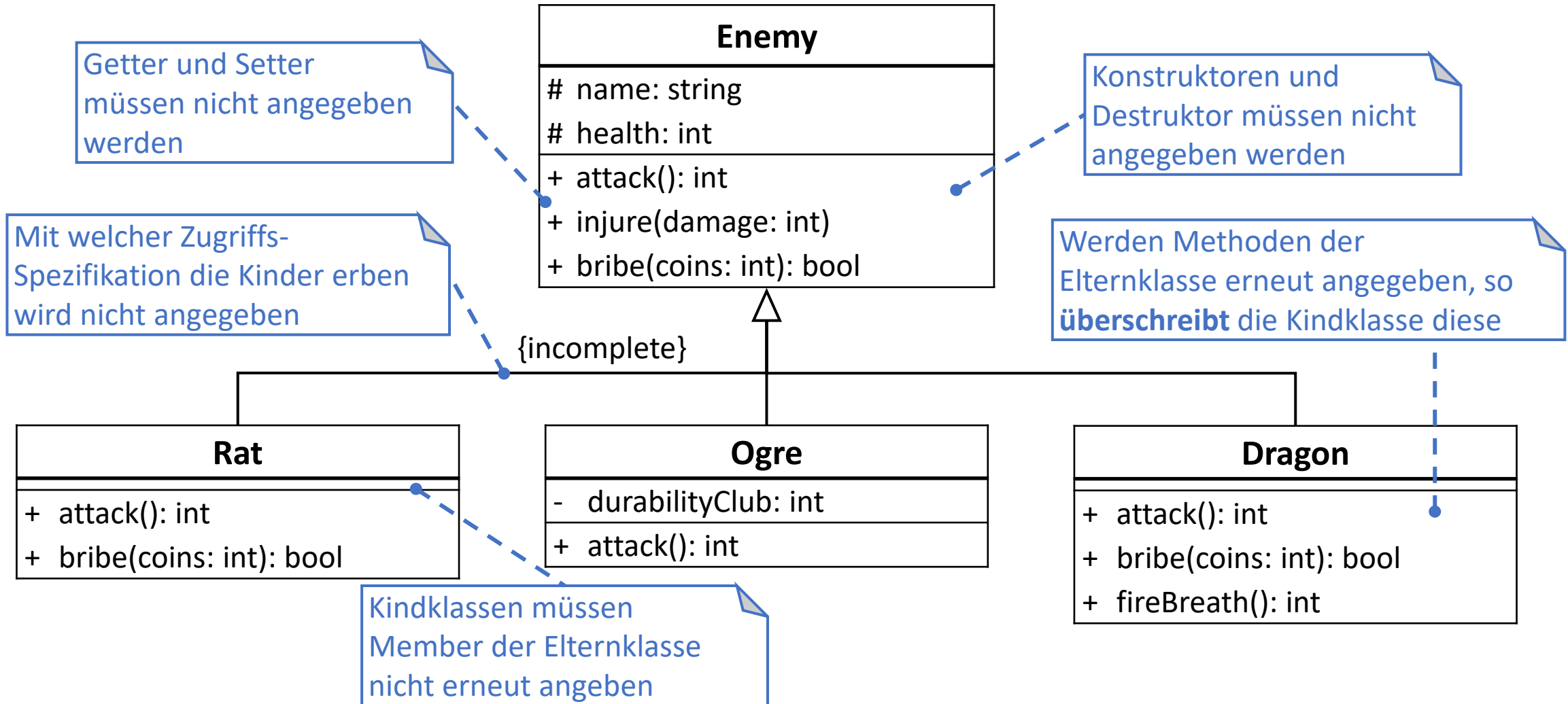
UML-Klassendiagramm

Gesamt



UML-Klassendiagramm

Weiter Erläuterungen



UML-Klassendiagramme

Umsetzung in C++



```
class Enemy
{
    // Konstruktor & Destruktor
public:
    explicit Enemy(std::string name, int health);

    // Getter & Setter
public:
    std::string getName() const;
    int getHealth() const;

public:
    virtual int attack();
    void injure(int damage);
    virtual bool bribe(int coins);

protected:
    std::string m_name;
    int m_health;
};
```

Enemy.h

virtual deklariert, welche Methoden von erbenden Klassen überschrieben werden dürfen.

Klassendiagramm übersichtlich halten, um Verständnis zu erzeugen und die Implementierung nicht einzuschränken.

Enemy

```
# name: string
# health: int
+ attack(): int
+ injure(damage: int)
+ bribe(coins: int): bool
```

Konvention: *m_* beim Namen, um die Variable als Member-Variable zu kennzeichnen und Verwechslungen zu vermeiden.
Muss im UML Diagramm nicht angegeben werden.

UML-Klassendiagramme

Umsetzung in C++



```
#include "Enemy.h"
```

Enemy.cpp

```
Enemy::Enemy(const std::string& name, int health)
```

```
    : m_name(name)
```

```
    , m_health(health)
```

```
{}
```

```
/* ... Getter ... */
```

```
int Enemy::attack() {
```

```
    // Verursache einen Schaden
```

```
    return 1;
```

```
}
```

```
void Enemy::injure(int damage) {
```

```
    // Erhalte Schaden
```

```
    m_health -= damage;
```

```
}
```

```
bool Enemy::bribe(int coins) {
```

```
    if (coins > 0)
```

```
        std::cout << m_name << ": Danke für das Geld!\n";
```

```
    return coins > 0;
```

```
}
```

Ohne *m_* müssten die Parameter anders benannt werden.

Mit *Enemy::* wird außerhalb der Klassendeklaration auf Member der Klasse / auf den Scope der Klasse zugegriffen.

Das UML-Klassendiagramm enthält keine konkrete Information über die Funktionalität der Klasse.

Enemy

name: string

health: int

+ attack(): int

+ injure(damage: int)

+ bribe(coins: int): bool

UML-Klassendiagramme

Umsetzung in C++

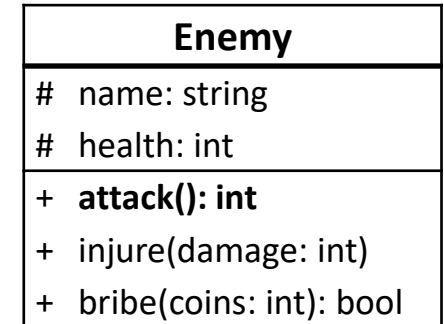


```
class Rat : public Enemy {  
public:  
    Rat(const std::string& name, int health);  
    int attack() override;  
    bool bribe(int coins) override;  
};
```

Rat.h

Konstruktoren der
Kindklassen geben
Parameter an die
Elternklasse weiter.

Die Header der Kindklassen
müssen den Header der
Elternklasse inkludieren!



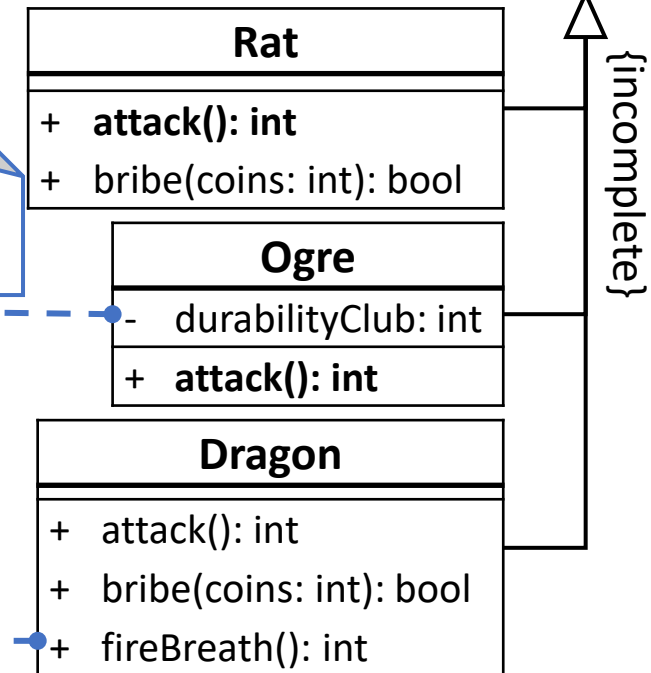
```
class Ogre : public Enemy {  
public:  
    Ogre(const std::string& name, int health, int durabilityClub);  
    int attack() override;  
private:  
    int m_durabilityClub;  
};
```

Ogre.h

override deklariert, dass eine
abgeleitete Methode
überschrieben wird. (optional)

```
class Dragon : public Enemy {  
public:  
    Dragon(const std::string& name, int health);  
    int attack() override;  
    bool bribe(int coins) override;  
    int fireBreath();  
};
```

Dragon.h



UML-Klassendiagramm

Zusammenfassung



- Ein **UML-Klassendiagramm** kann den entworfene Klassenaufbau visuell und kompakt darstellen.
 - Das UML-Klassendiagramm ist weitestgehend **Programmiersprachen-unabhängig**
 - In dem Diagramm werden die „**besonderen**“ **Funktionalitäten und Informationen** der Klassen und ihrer Beziehungen beschrieben
 - Getter und Setter sowie Konstruktoren und Destruktoren werden **nicht angegeben**
 - Member der Elternklasse werden automatisch an die Kindklassen vererbt, ohne, dass diese erneut gelistet werden müssen
 - Werden vererbte Methoden in der Kindklasse erneut angegeben, heißt das, dass eine Überschreibung stattfindet

UML-Klassendiagramm

Verständnisfragen

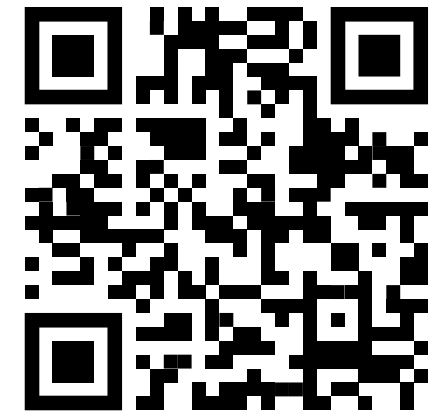


■ WAHR oder FALSCH?

- Ein UML-Klassendiagramm kann bei der Planung der Klassenarchitektur helfen.
- Ein UML-Klassendiagramm kann bei der Dokumentation der Klassenarchitektur helfen.
- Das UML-Klassendiagramm dient der vollständigen Dokumentation von implementierten Klassen und ihren Funktionen.
- Aus einem UML-Klassendiagramm sollte erkenntlich werden, in welcher Programmiersprache die Klassen implementiert sind.

PIN: 967140

<https://poll.hs-kempten.de/poll/>





Polymorphismus

Lernziele

- Der Begriff Polymorphismus wurde verstanden und kann erklärt werden
 - Methoden können überschrieben werden
 - Die Begriffe und der Nutzen von „virtuellen“ und „rein virtuellen Methoden“ ist verstanden
 - Das Prinzip von Virtual Tables ist bekannt
 - Abstrakte Klassen und Schnittstellenklassen können implementiert werden

Polymorphismus

Motivation



```
#include "Dragon.h"
#include "Rat.h"
#include "Ogre.h"
int main() {
    Enemy* creatures[3] {
        new Ogre{ "Grump", 10, 1 },
        new Dragon{ "Smaug", 200 },
        new Rat{ "Wurmschwanz", 1 }
    };
    for (int i = 0; i < 3; ++i) {
        std::cout << creatures[i]->getName() << " macht " << creatures[i]->attack();
        std::cout << " und " << creatures[i]->attack() << " Schaden." << '\n';
    }
    std::cout << '\n';
    for (int i = 0; i < 3; ++i) {
        if(!creatures[i]->bribe(10))
            creatures[i]->bribe(150);
    }
    for (int i = 0; i < 3; ++i)
        delete creatures[i];
}
```

- Häufig möchte man, dass sich die Implementierung von Funktionen der Elternklasse unter den Kindklassen **unterscheiden**
- Als Anwender möchten wir die Funktionen dennoch **unabhängig von der Spezialisierung** für alle Kindklassen **gleich** verwenden

Gewünschte Ausgabe:

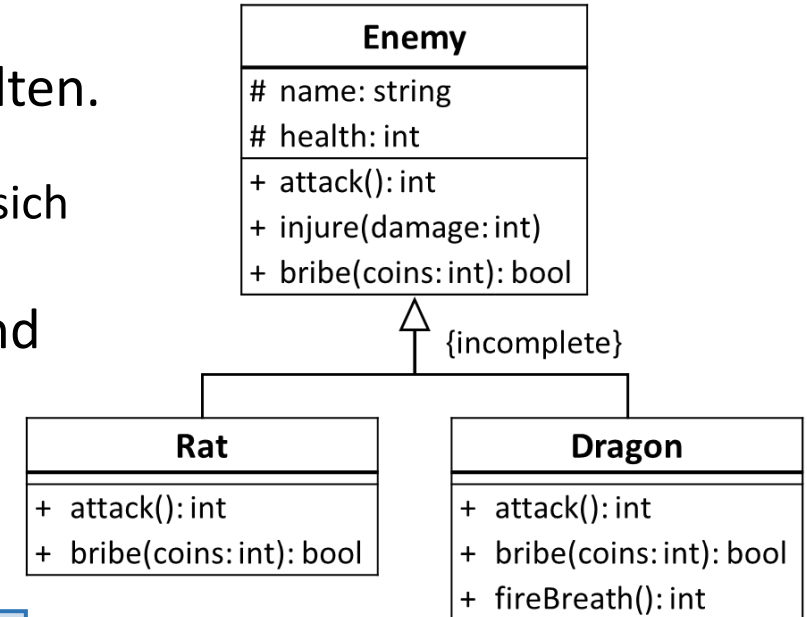
```
Grump macht 20 und 2 Schaden.
Smaug macht 30 und 30 Schaden.
Wurmschwanz macht 2 und 2 Schaden.

Grump: Danke fuer das Geld!
Smaug: Willst du mich mit diesen Almosen verspotten?
Smaug: Eine weise Entscheidung!
Wurmschwanz schnuppert kurz an dem Geld und ignoriert es dann.
Wurmschwanz schnuppert kurz an dem Geld und ignoriert es dann.
```

Polymorphismus

Definition

- **Polymorphismus** (gr. „Vielgestaltigkeit“) bezeichnet das Überschreiben von Methoden mit spezialisiertem Verhalten.
 - Der Anwender kann die Methode unabhängig von der Spezialisierung immer gleich aufrufen. Das Verhalten kann sich entsprechend der Kindklasse unterscheiden.
- Im Beispiel: Ratte und Drache **überschreiben** „attack“ und „bribe“ mit unterschiedlichem Verhalten.
 - Es kann für alle Gegner versucht werden, sie zu bestechen, aber welchen Effekt das hat, kann je nach Kindklasse unterschiedlich sein.



```
bool Dragon::bribe(int coins) {
    if (coins > 100) {
        std::cout << m_name << ": Eine weise Entscheidung!\n";
        return true;
    }
    std::cout << m_name << ": Willst du mich mit diesen Almosen verspotten?\n";
    return false;
}
```

Dragon.cpp

```
bool Rat::bribe(int coins) {
    std::cout << m_name
        << " schnuppert kurz an dem Geld"
        << " und ignoriert es dann.\n";
    return false;
}
```

Rat.cpp

Polymorphismus

Up-Cast



```
#include "Dragon.h"
#include "Rat.h"
#include "Ogre.h"
int main() {
    Enemy* creatures[3] {
        new Ogre{ "Grump", 10, 1 },
        new Dragon{ "Smaug", 200 },
        new Rat{ "Wurmschwanz", 1 }
    };

    std::cout << creatures[0]->getName()
        << " macht " << creatures[0]->attack()
        << " und " << creatures[0]->attack()
        << " Schaden.\n";
    creatures[1]->bribe(10);
    creatures[1]->bribe(150);
    creatures[2]->bribe(1);

    for (int i = 0; i < 3; ++i)
        delete creatures[i];
}
```

Ausgabe:

```
Grump macht 20 und 2 Schaden.
Smaug: Willst du mich mit diesen Almosen verspotten?
Smaug: Eine weise Entscheidung!
Wurmschwanz schnuppert kurz an dem Geld und ignoriert es dann.
```

- Eine Instanz der Kindklasse gilt immer auch implizit als eine Instanz der Elternklasse (**Up-Cast**)
 - Ein Pointer, der auf ein Objekt der Elternklasse deklariert wird, kann auch auf Instanzen der Kindklassen zeigen.
 - Die Funktionen, die in der Elternklasse definiert sind, können unabhängig von der Kindklasse aufgerufen werden.
 - Bei einer überschriebenen Funktion wird die Implementierung der Kindklasse aufgerufen. (**Polymorphismus**)

Polymorphismus



Virtual Tables

- Damit die korrekte Implementierung von virtuellen Methoden aufgerufen wird, existieren **Virtual Method Tables** (vtables).
 - Ist eine Methode **nicht virtual**, kann der Compiler die Funktion einmal für alle Instanzen der Klasse statisch verarbeiten (**static dispatch** / **early binding**).
 - Für **Virtual**-Methoden kann der Compiler nicht wissen, welche Implementierung der Methode zur Compile-Zeit ausgeführt werden soll. Der Compiler muss die korrekte Implementierung dynamisch zur Laufzeit verarbeiten (**dynamic dispatch** / **late method binding**).

```
Enemy* creatures[3] {  
    new Ogre{ "Grump",10,1 },  
    new Dragon{ "Smaug", 200 },  
    new Rat{ "Wurmschwanz",1 }  
};  
creatures[1]->bribe(10);  
creatures[1]->bribe(150);  
creatures[2]->bribe(1);
```

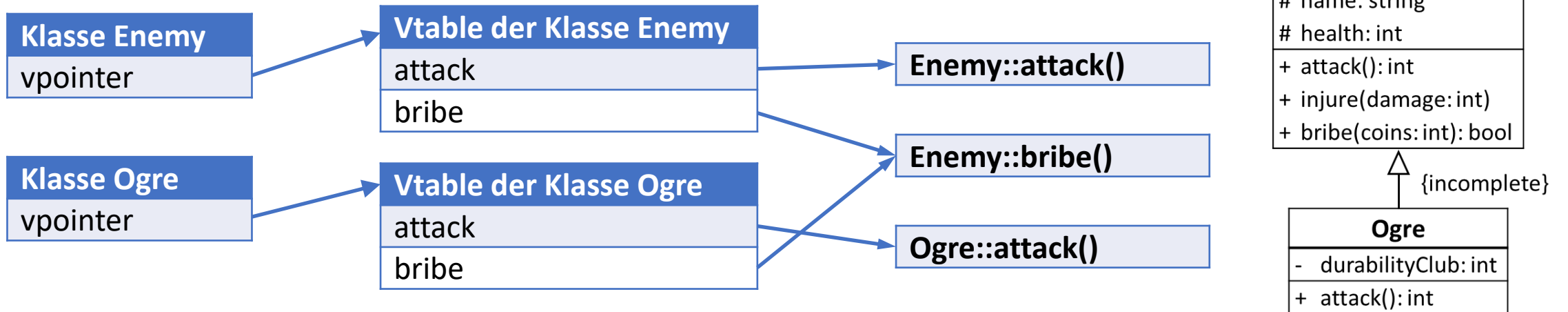
Ohne **dynamic dispatch** würde hier immer `Enemy::bribe()` aufgerufen werden und nicht `Drache::bestechen()` oder `Ratte::bestechen()` !

Polymorphismus

Virtual Tables



- Für jede Klasse, die eine virtuelle Funktion besitzt, wird ein **Virtual Table** erzeugt.
 - Der Virtual Table enthält für jede virtuelle Funktion in der Klasse einen **Pointer zu seiner spezialisiertesten Definition**.
 - Damit zeigen die Pointer immer zu der spezifischen, überschriebenen Funktion in der eigenen Kindklasse oder zu der vererbten Funktion aus der Elternklasse.
 - Damit die Virtual Table der entsprechenden Klasse zugeordnet werden kann, wird vom Compiler für jede Klasse ein sogenannter **vpointer** als zusätzliche Membervariable hinzugefügt, der auf den korrekten Virtual Table zeigt.



Polymorphismus

Virtueller Destruktor



- Besitzt eine Kindklasse einen Destruktor, so **muss** in der Elternklasse ein **virtueller Destruktor** vorhanden sein, um zu gewährleisten, dass der Destruktor der Kindklasse immer aufgerufen wird.
 - Ohne virtuellen Destruktor, wird der Destruktor der Elternklasse statisch verarbeitet.
 - **Best Practice:** Immer einen virtuellen Destruktor in der Elternklasse deklarieren

```
class Enemy
{
    // Konstruktor & Destruktor
    public:
        explicit Enemy(std::string name, int health);

        virtual ~Enemy();

    /* ... */
};
```

Polymorphismus



Virtueller Destruktor – Beispiel

- Wird in der Elternklasse **kein** oder ein **nicht-virtueller Destruktor** deklariert, so wird für den Destruktor auch kein Eintrag im Virtual Table erstellt.
 - Der Destruktor der Kindklasse wird **nicht aufgerufen!**

```
class Base {  
public:  
    virtual ~Base() { std::cout << "Destroying base\n"; }  
};
```

```
int main() {  
    Base* p = new Derived(5);  
    delete p;  
}
```

Ausgabe **ohne virtual**:

```
Destroying base
```

Ausgabe **mit virtual**:

```
Destroying derived  
Destroying base
```

```
class Derived : public Base {  
public:  
    Derived(int number)  
        : m_element{ new int(number) } {}  
    ~Derived() {  
        std::cout << "Destroying derived\n";  
        delete m_element;  
    }  
private:  
    int* m_element;  
};
```

Polymorphismus

Abstrakte Klassen

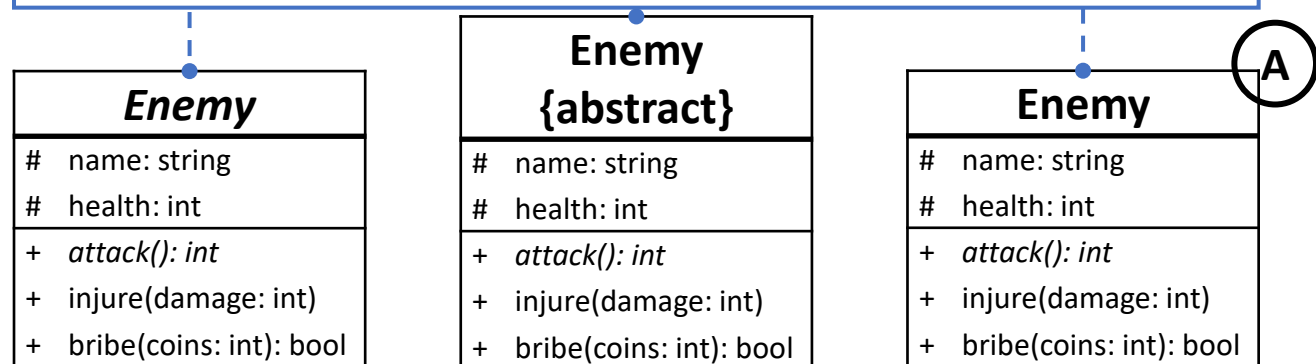


- Eine Methode, die nur deklariert, aber nicht definiert wird, ist eine **rein virtuelle Methode** und wird mit **= 0** am Ende der Deklaration gekennzeichnet
 - Klassen, die mindestens eine rein virtuelle Methode besitzen, sind **abstrakt**
 - Von abstrakten Klassen können keine Instanzen erstellt werden
 - Der Destruktor kann nicht rein virtuell sein!
 - Rein virtuelle Methoden zwingen Kindklassen dazu, diese zu überschreiben, um instanziiert werden zu können

```
class Enemy {  
    // ... Konstruktoren, Getter & Setter, ...  
public:  
    virtual int attack() = 0;  
    void injure(int damage);  
    virtual bool bribe(int coins);  
protected:  
    std::string m_name;  
    int m_health;  
};
```

Abstrakte Klasse in UML:

1. Möglichkeit: Kursiver Klassenname
2. Möglichkeit: Klassenname wird mit {abstract} vermerkt
3. Möglichkeit: Ein eingekreistes A wird zur Klasse geschrieben.



Polymorphismus

Zusammenfassung



- **Polymorphismus** ermöglicht, dass Kindklassen eine gemeinsame, vererbte Fähigkeit unterschiedlich umsetzen können.
 - Um virtuelle Funktionen umzusetzen, erzeugt der Compiler **Virtual Tables**.
 - Um sicherzustellen, dass alle Instanzen auch ordentlich wieder zerstört werden, muss der **Destruktor** in der Elternklasse als **virtuell** gekennzeichnet werden.
 - Ist die Umsetzung für jede Kindklasse einzigartig, also gibt es kein „Default-Verhalten“, so kann die Funktion in der Elternklasse als **rein virtuell** deklariert werden.

Polymorphismus

Verständnisfragen

PIN: 967140

<https://poll.hs-kempten.de/poll/>



■ Wofür nutzt man das Schlüsselwort `virtual` in C++?

- Gibt es in C++ nicht.
- Zur Deklaration von Funktionszeigern.
- Um eine Member-Funktion von erbbenden Klassen überschreiben zu können.
- Spezifiziert, welche Member-Variablen erst durch erbende Klassen initialisiert werden.

■ WAHR oder FALSCH?

- Polymorphismus ermöglicht, Objekte verschiedener Klassen auf dieselbe Weise zu behandeln.
- Polymorphismus ermöglicht es, eine Funktion auf unterschiedliche Arten zu implementieren.
- Polymorphismus wird verwendet, um die Erzeugung von Virtual Method Tables zu umgehen.
- Instanzen einer Kindklasse können nur die Methoden der Elternklasse aufrufen, die mit **virtual** deklariert sind.
- Wenn bei Vererbung der Destruktor der Elternklasse **nicht** als **virtual** deklariert wird, kann es vorkommen, dass der Destruktor für ein Objekt der Kindklasse **nicht** aufgerufen wird.
- Von Klassen, die virtuelle Methoden deklarieren, können keine Instanzen erstellt werden.
- Spezialisierungen einer abstrakten Klasse müssen rein virtuelle Funktionen überschreiben, um instanziiert werden zu können.
- Die Basisklasse definiert die Implementation einer rein virtuellen Funktion.



Down-Casts

Lernziele

- Der **Nutzen** und die **Notwendigkeit** von Down-Casts sind bekannt
- Down-Casts können verwendet werden, um einen **Verweis** auf eine Elternklasse in einen Verweis in eine Kindklasse **umzuwandeln**
- Die **Gefahren** und **Herausforderungen** von Downs-Casts sind verinnerlicht
- **Unterschiedliche Möglichkeiten** für Down-Casts mit ihren **Vor- und Nachteilen** sind bekannt



Down-Casts

Definition & Beispiel

```
Enemy* creatures[3] {  
    new Ogre{ "Grump", 10, 1 },  
    new Dragon{ "Smaug", 200 },  
    new Rat{ "Wurmschwanz", 1 }  
};
```

```
creatures[1]->fireBreath();
```

```
for (int i = 0; i < 3; ++i)  
    delete creatures[i];
```

```
int main()
```

„Klasse „**Enemy**“ hat keinen Member „**fireBreath**“.

- Methoden der Kindklassen, die nicht in der Elternklasse definiert sind, können **nicht** über die Elternklasse aufgerufen werden. (**Down-Cast**)

- Selbst, wenn der Pointer oder die Referenz auf eine Instanz der Elternklasse mit einer entsprechenden Instanz der Kindklasse initialisiert wurde.
- Dazu benötigt es eine **explizite Typumwandlung**

Dragon
+ attack(): int
+ bribe(coins: int): bool
+ fireBreath(): int



Down-Casts

Umwandlungsmöglichkeiten

- **C-style Cast**

Umwandlungsoperator für alle Arten von Umwandlungen im C-Format

- **Static Cast**

Für Umwandlungen zwischen numerischen Datentypen und Enumerationen.
Und für explizite Up- und Down- Casts in Klassenhierarchien ohne Überprüfung zur Laufzeit.

- **Dynamic Cast**

Für explizite Up- und Down- Casts in Klassenhierarchien mit Überprüfung zur Laufzeit, ob die Umwandlung valide ist.



Down-Casts

C-style Cast

- Der C-style Cast funktioniert auch für Down-Casts, sollte hier aber vor allem **vermieden werden**
 - Ist im Code schwer zu erkennen
 - Sehr schwer vorhersehbar, was bei einer Umwandlung tatsächlich geschieht
 - Besitzt keine Überprüfung, ob der Down-Cast überhaupt korrekt ist

```
int main()
```

```
Dragon smaug{ "Smaug", 200 };  
Enemy& creature = smaug;  
((Dragon&)creature).fireBreath();
```

BITTE NICHT VERWENDEN!

Ausgabe:

```
Smaug speiht Feuer und entflammt die Umgebung.
```

Down-Casts



Static Cast

- Mit ***static_cast*** werden Umwandlungen zur Kompilierungszeit überprüft
 - Gibt einen Fehler zurück, wenn der Compiler erkennt, dass zwischen inkompatiblen Typen umgewandelt werden soll
- **Down-Casts** von Objekten der Basisklasse auf Objekte der abgeleiteten Klasse sind möglich
 - Der Compiler kann nicht immer erkennen, ob die Umwandlung auch zur Laufzeit sicher ist
 - Deshalb muss der **Programmierer** gewährleisten, dass die Umwandlung sicher ist

```
double d = 3.1415925;
std::string s = static_cast<std::string>(d); // ERROR C2440: "static_cast":
// double kann nicht in "std::string" konvertiert werden

Dragon smaug{ "Smaug", 200 };
Enemy& creature = smaug;
static_cast<Dragon&>(creature).fireBreath(); // Funktioniert
```

```
int main()
```

Down-Casts

Dynamic Cast



- Bei **Dynamic Casts** wird die Umwandlung **zur Laufzeit** auf Typsicherheit überprüft
 - Zur sicheren Umwandlung von Pointern und Referenzen auf Klassen innerhalb der gesamten Vererbungshierarchie
 - Verursacht dadurch einen Mehraufwand zur Laufzeit
 - Bei fehlerhafter Pointer-Umwandlung wird ein **Nullpointer** zurückgegeben
 - Bei fehlerhafter Referenz-Umwandlung wird ein **Laufzeitfehler** geworfen

```
Enemy* creatures[3] {new Ogre{ "Grump",10,1 }, new Dragon{ "Smaug", 200}, new Rat{ "Wurmschwanz",1 }};
for (int i = 0; i < 3; ++i)
{
    if (auto dragon = dynamic_cast<Dragon*>(creatures[i]); dragon)
        std::cout << dragon->getName() << " ist ein Dragon.\n";
    else
        std::cout << creatures[i]->getName() << " ist kein Dragon.\n";
}
for (int i = 0; i < 3; ++i)
    delete creatures[i];
```

```
int main()
```

```
Grump ist kein Drache.
Smaug ist ein Drache.
Wurmschwanz ist kein Drache.
```



Down-Casts

Vergleich der Umwandlungsmöglichkeiten

```
// Includes
int main() {
    Enemy* creatures[3] {
        new Ogre{ "Grump", 10, 1 },
        new Dragon{ "Smaug", 200 },
        new Rat{ "Wurmschwanz", 1 }
    };
}
```

- a) `for (int i = 0; i < 3; ++i)`
 `((Dragon*)creatures[i])->fireBreath();`
- b) `for (int i = 0; i < 3; ++i)`
 `static_cast<Dragon*>(creatures[i])->fireBreath();`
- c) `for (int i = 0; i < 3; ++i)`
 `if(auto dragon = dynamic_cast<Dragon*>(creatures[i]); dragon)`
 `dragon->fireBreath();`
- `for (int i = 0; i < 3; ++i)`
 `delete creatures[i];`
`}`

Überlegt Euch, was bei jedem der drei Varianten geschehen könnte, wenn sie so in den Code eingefügt werden und das Programm gestartet wird.

Down-Casts

Vergleich der Umwandlungsmöglichkeiten



```
// Includes
int main() {
    Enemy* creatures[3] {
        new Ogre{ "Grump", 10, 1 },
        new Dragon{ "Smaug", 200 },
        new Rat{ "Wurmschwanz", 1 }
    };
}
```

a) `for (int i = 0; i < 3; ++i)`
`((Dragon*)creatures[i])->fireBreath();`

b) `for (int i = 0; i < 3; ++i)`
`static_cast<Dragon*>(creatures[i])->fireBreath();`

c) `for (int i = 0; i < 3; ++i)`
`if(auto dragon = dynamic_cast<Dragon*>(creatures[i]))`
`dragon->fireBreath();`

```
for (int i = 0; i < 3; ++i)
    delete creatures[i];
}
```

Das Verhalten von **a) und b)** ist **nicht definiert**.

Das Programm kann funktionieren und die anderen Gegner-Typen wie einen Drachen behandeln oder abstürzen.

In diesem Fall läuft das Programm mit der Ausgabe:

```
Grump speiht Feuer und entflammt die Umgebung.
Smaug speiht Feuer und entflammt die Umgebung.
Wurmschwanz speiht Feuer und entflammt die Umgebung.
```



Down-Casts

Vergleich der Umwandlungsmöglichkeiten

```
// Includes
int main() {
    Enemy* creatures[3] {
        new Ogre{ "Grump", 10, 1 },
        new Dragon{ "Smaug", 200 },
        new Rat{ "Wurmschwanz", 1 }
    };
}
```

a)

```
for (int i = 0; i < 3; ++i)
    ((Dragon*)creatures[i])->fireBreath();
```

b)

```
for (int i = 0; i < 3; ++i)
    static_cast<Dragon*>(creatures[i])->fireBreath();
```

c)

```
for (int i = 0; i < 3; ++i)
    if(auto dragon = dynamic_cast<Dragon*>(creatures[i]); dragon)
        dragon->fireBreath();
```

```
for (int i = 0; i < 3; ++i)
    delete creatures[i];
}
```

`fireBreath()` wird nur für die Objekte aufgerufen, bei denen der Down-Cast **typsicher** ist:

Smaug speiht Feuer und entflammt die Umgebung.

Down-Casts

Programmabsturz bei unsicherer Typumwandlung



Dragon.cpp

```
int Dragon::fireBreath() {  
    std::cout << m_name << " speiht Feuer und entflammt " << m_buddy.getName() << ".\n";  
    return 20;  
}
```

```
int main() {  
    Enemy* creatures[3] {  
        new Ogre{ "Grump", 10, 1 },  
        new Dragon{ "Smaug", 200 },  
        new Rat{ "Wurmschwanz", 1 }  
    };  
}
```

b)

```
for (int i = 0; i < 3; ++i)  
    static_cast<Dragon*>(creatures[i])->fireBreath();
```

```
for (int i = 0; i < 3; ++i)  
    delete creatures[i];  
}
```

Dragon
- buddy: Enemy&
+ attack(): int
+ bribe(coins:int): bool
+ fireBreath(): int

Ist die aufzurufende Funktion abhängig von einem Objekt, welches zwingend initialisiert werden muss, stürzt das Programm beim versuchten Zugriff darauf ab:

Ausgelöste Ausnahme: Lesezugriffsverletzung
"this" war "0xFFFFFFFFFFFFFFE7".

Konsole: Grump speiht Feuer und entflammt

Down-Casts



Zusammenfassung

- Will man von einem Pointer/einer Referenz einer Elternklasse eine Methode einer speziellen Kindklasse ausführen, benötigt man **Down-Casts**.
 - Dies kann sehr **fehleranfällig** sein.
 - Eine fehlerhafte Typumwandlung führt zu **undefiniertem Verhalten**
- Der **C-Style-Cast** sollte unter allen Umständen **vermieden werden**
- Der **Static Cast** überprüft nicht, ob eine Umwandlung zur Laufzeit sicher ist
 - Der Static Cast besitzt zur Laufzeit keinen Overhead
 - Sollte nur verwendet werden, wenn der Programmierer sicherstellen kann, dass die Umwandlung zur Laufzeit sicher ist
- Der **Dynamic Cast** überprüft zur **Laufzeit**, ob die Umwandlung **typsicher** ist
 - Dadurch wird zur Laufzeit ein Mehraufwand erzeugt
 - Bei fehlerhafter Pointer-Umwandlung wird ein **Nullpointer** zurückgegeben
 - Bei fehlerhafter Referenz-Umwandlung wird ein **Laufzeitfehler** geworfen

Down-Casts

Verständnisfragen

PIN: 967140

<https://poll.hs-kempten.de/poll/>



■ Welche Typumwandlungen sind typsicher?

- C-Style Cast
- Static Cast
- Const Cast
- Dynamic Cast

■ WAHR oder FALSCH?

- Up-Casts sind immer typsicher.
- Der Dynamic Cast überprüft zur Kompilierungszeit, ob eine Typumwandlung sicher ist.
- Der Static Cast überprüft zur Kompilierungszeit, ob eine Typumwandlung zwischen zwei Typen unmöglich ist.
- Ein Static Cast verhält sich für alle Anwendungsfälle gleich wie der C-style Cast.
- Eine unsichere Typumwandlung führt immer zum Programmabsturz.